

UNIVERSIDADE FEDERAL DE SANTA MARIA

COLÉGIO POLITÉCNICO

CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

BRUNO FETZER COSTA DA ROSA

BRUNO VALCARENGHI MACHADO

DAVI FROEMMING DE MENEZES

FREDERICO BRIGIDO DE OLIVEIRA

ODORES EM CÓDIGO-FONTE E REFATORAÇÃO

SANTA MARIA

2026

BRUNO FETZER COSTA DA ROSA
BRUNO VALCARENGHI MACHADO
DAVI FROEMMING DE MENEZES
FREDERICO BRIGIDO DE OLIVEIRA

ODORES EM CÓDIGO-FONTE E REFATORAÇÃO

Trabalho apresentado à disciplina de Engenharia de Software do Curso Superior de Tecnologia em Sistemas para Internet do Colégio Politécnico da Universidade Federal de Santa Maria.
Professor: Dr. Henry Cagnini.

SANTA MARIA
2026

RESUMO

Este trabalho apresenta os odores em código-fonte como indícios de possíveis problemas estruturais e discute a refatoração como forma de melhorar a organização interna do software sem alterar seu comportamento observável. Com base na taxonomia do Refactoring.Guru e em referências de Engenharia de Software, descrevem-se 23 odores distribuídos em cinco categorias: Bloaters, Object-Orientation Abusers, Change Preventers, Dispensables e Couplers. Para cada odor, são sintetizados seus sinais, efeitos na manutenção e técnicas de refatoração relacionadas. Também se examinam o processo seguro de refatoração, o papel dos testes automatizados e os critérios para decidir quando intervir no código. Conclui-se que a presença de um odor não prova a existência de defeito: sua relevância depende do contexto e das necessidades de evolução. A refatoração deve ser aplicada em passos verificáveis, com atenção ao custo e à preservação do comportamento. A análise também considera coesão, acoplamento, frequência de mudanças e riscos de manutenção para orientar a escolha contextual de cada transformação.

Palavras-chave: Odores de código; refatoração;
qualidade de software; manutenção de software.

SUMÁRIO

1 INTRODUÇÃO 5

2 ODORES EM CÓDIGO-FONTE 6

2.1 Bloaters 6

2.1.1 Long Method 6

2.1.2 Large Class 6

2.1.3 Primitive Obsession 7

2.1.4 Long Parameter List 7

2.1.5 Data Clumps 7

2.2 Object-Orientation Abusers 7

2.2.1 Switch Statements 7

2.2.2 Temporary Field 7

2.2.3 Refused Bequest 8

2.2.4 Alternative Classes with Different Interfaces 8

2.3 Change Preventers 8

2.3.1 Divergent Change 8

2.3.2 Shotgun Surgery 8

2.3.3 Parallel Inheritance Hierarchies 8

2.4 Dispensables 9

2.4.1 Comments 9

2.4.2 Duplicate Code 9

2.4.3 Lazy Class 9

2.4.4 Data Class 9

2.4.5 Dead Code 10

2.4.6 Speculative Generality 10

2.5 Couplers 10

2.5.1 Feature Envy 10

2.5.2 Inappropriate Intimacy 10
2.5.3 Incomplete Library Class 10
2.5.4 Message Chains 11
2.5.5 Middle Man 11
3 REFATORAÇÃO 12
3.1 Relação entre odores e refatorações 12
3.2 Processo seguro de refatoração 12
3.3 Principais grupos de técnicas 13
3.4 Exemplo de refatoração: Extract Method 14
3.5 Testes, ferramentas e análise estática 14
3.6 Quando refatorar e quando não refatorar 15
4 METODOLOGIA PRÁTICA: ATIVIDADE DESPLUGADA 16
4.1 Objetivo da atividade 16
4.2 Materiais utilizados 16
4.3 Dinâmica e regras de execução 16
5 CONCLUSÃO 17
REFERÊNCIAS 18

1 INTRODUÇÃO

Um software profissional não se limita ao código-fonte: ele compreende o programa e a documentação associada (Cagnini, s. d.). No escopo deste trabalho, consideram-se também modelos, dados de configuração, testes e outros artefatos necessários à sua operação e evolução. A Engenharia de Software organiza métodos e práticas para conduzir esse ciclo de vida, desde a especificação até a manutenção. Nesse contexto, a qualidade não depende apenas do funcionamento imediato do sistema, mas também de atributos como confiabilidade, usabilidade e manutenibilidade (Sommerville, 2011).

Sistemas de software mudam continuamente. Requisitos são revistos, regras de negócio evoluem, tecnologias são substituídas e defeitos precisam ser corrigidos. Quando a estrutura interna do código não acompanha essa evolução, pequenas alterações passam a exigir esforço desproporcional, elevando o risco de regressões. A identificação de odores em código-fonte e a aplicação criteriosa de refatorações constituem práticas que ajudam a controlar esse processo.

Odores de código, ou code smells, são indícios superficiais de que pode existir um problema estrutural mais profundo. Eles não correspondem necessariamente a erros de compilação ou defeitos observáveis e não justificam, por si sós, uma mudança. Fowler (2006) destaca que um odor deve funcionar como ponto de partida para investigação: somente a análise do contexto permite concluir se a estrutura prejudica a compreensão, a manutenção ou a evolução do sistema.

A refatoração consiste na alteração da estrutura interna do software para torná-lo mais fácil de compreender e modificar, preservando seu comportamento observável (Fowler, 2004). Este trabalho apresenta as cinco categorias de odores adotadas pelo Refactoring.Guru e descreve os 23 odores reunidos nessa taxonomia (Refactoring.Guru, s. d.-a). Também relaciona sua identificação a técnicas de refatoração, testes e análise contextual.

2 ODORES EM CÓDIGO-FONTE

O termo code smell foi difundido por Kent Beck e Martin Fowler para descrever características do código que podem sinalizar dificuldades de projeto. O conceito é deliberadamente diagnóstico: um método extenso, uma classe com muitos campos ou uma cadeia de chamadas não constituem problemas em todos os casos. A avaliação deve considerar a frequência de mudanças, o domínio, a clareza da intenção, o nível de acoplamento e o custo de manutenção.

A taxonomia utilizada neste trabalho organiza os odores em cinco grupos: Bloaters, Object-Orientation Abusers, Change Preventers, Dispensables e Couplers. A classificação facilita o reconhecimento de padrões recorrentes, mas as categorias não são isoladas. Um mesmo trecho pode apresentar vários odores, e a correção de um problema pode revelar ou até introduzir outro se a refatoração for aplicada mecanicamente (Refactoring.Guru, s. d.-a).

2.1 Bloaters

Bloaters reúnem estruturas que cresceram gradualmente até se tornarem grandes demais para serem compreendidas ou modificadas com facilidade. Em geral, refletem acúmulo de responsabilidades, dados mal modelados ou ausência de decomposição adequada.

2.1.1 Long Method

Ocorre quando um método concentra muitas etapas, decisões ou responsabilidades. O número de linhas é apenas um indício; o problema principal é a dificuldade de compreender a intenção, testar partes isoladamente e alterar uma etapa sem afetar as demais. Extract Method e Replace Method with Method Object estão entre as técnicas que podem reduzir esse odor (Refactoring.Guru, s. d.-a).

2.1.2 Large Class

Caracteriza classes que reúnem muitos campos, métodos e motivos de mudança. A baixa coesão costuma aparecer quando grupos de atributos são usados por subconjuntos diferentes de métodos. Extract Class, Extract Subclass e Extract Interface ajudam a redistribuir responsabilidades quando essa separação corresponde ao domínio (Refactoring.Guru, s. d.-a).

2.1.3 Primitive Obsession

Surge quando conceitos relevantes do domínio, como dinheiro, telefone, endereço ou intervalo, são representados apenas por números e textos. Essa escolha espalha validações e regras pelo sistema. Replace Data Value with Object e Replace Type Code with Class permitem aproximar os dados de seus comportamentos (Refactoring.Guru, s. d.-a).

2.1.4 Long Parameter List

Listas extensas de parâmetros dificultam a leitura das chamadas e aumentam a chance de troca de argumentos. Quando os mesmos valores circulam juntos, Introduce Parameter Object ou Preserve Whole Object pode explicitar a relação entre eles. Ainda assim, criar um objeto apenas para reduzir uma contagem numérica não é justificativa suficiente (Refactoring.Guru, s. d.-a).

2.1.5 Data Clumps

Ocorre quando o mesmo conjunto de dados aparece repetidamente em campos ou parâmetros. A persistência do grupo sugere que ele representa um conceito ausente no modelo. A extração para uma classe própria reduz repetição e cria um local adequado para regras relacionadas (Refactoring.Guru, s. d.-a).

2.2 Object-Orientation Abusers

Esta categoria reúne aplicações incompletas ou inadequadas de princípios de orientação a objetos. Os indícios envolvem condicionais repetidas, estados temporários, hierarquias incompatíveis e interfaces inconsistentes.

2.2.1 Switch Statements

Estruturas switch ou cadeias de if podem constituir odor quando a mesma decisão por tipo se repete em vários pontos e precisa ser alterada a cada nova variante. Replace Conditional with Polymorphism é uma possibilidade, mas condicionais pequenas, estáveis e localizadas podem ser mais simples do que uma hierarquia de classes (Refactoring.Guru, s. d.-a).

2.2.2 Temporary Field

Campos temporários recebem valores apenas durante uma operação específica e permanecem vazios ou sem significado no restante do ciclo de vida do objeto. Isso dificulta a

compreensão de invariantes. Extract Class ou Replace Method with Method Object pode isolar o contexto temporário (Refactoring.Guru, s. d.-a).

2.2.3 Refused Bequest

A herança recusada ocorre quando uma subclasse não utiliza ou não consegue cumprir parte do contrato herdado. Métodos sobrescritos apenas para lançar exceções são um sinal expressivo. Push Down Method ou Replace Inheritance with Delegation pode produzir uma relação mais coerente (Refactoring.Guru, s. d.-a).

2.2.4 Alternative Classes with Different Interfaces

Classes alternativas com interfaces diferentes executam funções equivalentes, mas expõem métodos incompatíveis. A padronização por Rename Method, Move Method, Extract Superclass ou uma interface comum pode permitir substituição transparente entre implementações (Refactoring.Guru, s. d.-a).

2.3 Change Preventers

Change Preventers indicam estruturas que tornam modificações mais amplas ou dispersas do que o necessário. O custo aparece quando uma alteração lógica obriga a editar elementos sem uma organização clara.

2.3.1 Divergent Change

A mudança divergente ocorre quando uma classe precisa ser alterada por razões independentes, como mudanças em persistência, apresentação e regras financeiras. Extract Class ajuda a separar eixos de mudança, aumentando a coesão (Refactoring.Guru, s. d.-a).

2.3.2 Shotgun Surgery

Na cirurgia de espingarda, uma única regra está espalhada por vários componentes; por isso, uma alteração exige pequenas edições em muitos locais. Move Method, Move Field ou Inline Class pode reunir a responsabilidade. Centralizar a regra não significa concentrar todo o sistema em uma classe (Refactoring.Guru, s. d.-a).

2.3.3 Parallel Inheritance Hierarchies

Hierarquias de herança paralelas aparecem quando a criação de uma subclasse obriga a criar outra correspondente em uma segunda hierarquia. Move Method e Move Field podem

reduzir a dependência, enquanto composição e delegação oferecem alternativas quando as hierarquias não representam uma relação estável (Refactoring.Guru, s. d.-a).

2.4 Dispensables

Dispensables são elementos cuja remoção tende a tornar o código mais direto. A exclusão, porém, deve ocorrer somente após a confirmação de que não há dependências, contratos ou decisões históricas relevantes.

2.4.1 Comments

Comentários tornam-se odor quando tentam compensar código confuso, repetem literalmente uma operação ou permanecem desatualizados. Antes de acrescentar uma explicação, convém avaliar Rename Method e Extract Method. Segundo Refactoring.Guru (s. d.-a), comentários que registram decisões, restrições externas ou razões não evidentes continuam úteis.

2.4.2 Duplicate Code

Código duplicado aumenta o risco de correções inconsistentes. A duplicação pode ser literal ou semântica, quando implementações diferentes resolvem o mesmo problema. Segundo Refactoring.Guru (s. d.-a), Extract Method, Pull Up Method e Form Template Method são alternativas, desde que a abstração resultante expresse uma regra realmente compartilhada.

2.4.3 Lazy Class

Uma classe preguiçosa acrescenta estrutura sem responsabilidade suficiente para justificar sua existência. Ela pode ser incorporada a outra classe por Inline Class. Entretanto, componentes pequenos podem ser válidos quando representam fronteiras arquiteturais ou conceitos importantes (Refactoring.Guru, s. d.-a).

2.4.4 Data Class

Classes de dados expõem campos por getters e setters, mas transferem o comportamento correspondente a outros objetos. Move Method e Encapsulate Field podem aproximar regras e dados. Segundo Refactoring.Guru (s. d.-a), objetos de transferência, registros imutáveis e modelos de integração são exceções comuns e não devem ser tratados automaticamente como falhas.

2.4.5 Dead Code

Código morto abrange variáveis, parâmetros, métodos, classes e ramos condicionais que não são executados. Depois de confirmar a ausência de uso por análise estática e testes, sua remoção reduz o volume de código e evita interpretações equivocadas sobre funcionalidades obsoletas (Refactoring.Guru, s. d.-a).

2.4.6 Speculative Generality

A generalidade especulativa surge quando abstrações e extensões são criadas para necessidades futuras não confirmadas. A estrutura aumenta o custo presente sem benefício comprovado. Collapse Hierarchy, Inline Class e Remove Parameter ajudam a recuperar uma solução orientada à necessidade atual (Refactoring.Guru, s. d.-a).

2.5 Couplers

Couplers tratam tanto do acoplamento excessivo quanto do excesso de delegação usado para evitá-lo. O objetivo não é eliminar dependências, mas mantê-las explícitas, estáveis e coerentes com as responsabilidades.

2.5.1 Feature Envy

Inveja de recurso ocorre quando um método utiliza mais dados de outro objeto do que da própria classe. Move Method pode aproximar comportamento e dados, respeitando a regra de colocar junto aquilo que muda junto. Métodos que coordenam vários objetos exigem análise adicional (Refactoring.Guru, s. d.-a).

2.5.2 Inappropriate Intimacy

A intimidade inapropriada aparece quando classes dependem de detalhes internos umas das outras. Encapsulate Field, Move Method, Move Field e a redução de associações bidirecionais ajudam a restaurar limites claros (Refactoring.Guru, s. d.-a).

2.5.3 Incomplete Library Class

Uma classe de biblioteca incompleta não oferece uma operação necessária ao sistema, e seu código não pode ser modificado diretamente. Introduce Foreign Method atende a uma necessidade pequena e localizada; Introduce Local Extension é mais adequada quando várias operações complementares precisam ser reunidas (Refactoring.Guru, s. d.-a).

2.5.4 Message Chains

Cadeias de mensagens expõem ao cliente uma sequência de relacionamentos, como objeto.getA().getB().getC(). Hide Delegate reduz o conhecimento da estrutura interna. Seu uso excessivo, contudo, pode criar intermediários sem valor, exigindo equilíbrio (Refactoring.Guru, s. d.-a).

2.5.5 Middle Man

O intermediário surge quando uma classe apenas encaminha a maior parte das solicitações para outro objeto. Remove Middle Man ou Inline Method simplifica a navegação, mas a delegação deve ser mantida quando protege uma fronteira, estabiliza uma API ou aplica uma regra relevante (Refactoring.Guru, s. d.-a).

3 REFATORAÇÃO

Refatorar não significa acrescentar funcionalidades nem reescrever o sistema sem restrições. Trata-se de uma transformação controlada que melhora a estrutura interna e preserva o comportamento observável. Essa distinção permite separar duas atividades: primeiro, alterar o projeto do código; depois, quando necessário, implementar um novo requisito sobre uma base mais compreensível (Fowler, 2004; Fowler, 2020).

O processo deve ocorrer em passos pequenos. Cada alteração precisa ter objetivo claro, ser validada por testes e permanecer simples de revisar ou reverter. Refatorações extensas realizadas de uma só vez dificultam a identificação da causa de uma regressão e aproximam o trabalho de uma reescrita. O Refactoring.Guru (s. d.-b) recomenda executar mudanças incrementais e testar após cada etapa.

3.1 Relação entre odores e refatorações

Um odor orienta a investigação, enquanto uma técnica de refatoração descreve uma transformação possível. Não existe correspondência única entre ambos. Long Method pode ser tratado com Extract Method, mas variáveis fortemente dependentes podem exigir Replace Method with Method Object. Feature Envy frequentemente sugere Move Method, embora o método possa coordenar dados de várias classes e permanecer melhor no local original.

A decisão deve avaliar coesão, acoplamento, frequência de mudança, clareza do domínio e custo da abstração. Uma transformação que elimina um odor pode introduzir outro: Hide Delegate reduz Message Chains, mas o excesso de métodos delegadores pode criar Middle Man. Por isso, a melhoria deve ser verificada no conjunto do projeto, e não apenas pela ausência de um sintoma isolado.

3.2 Processo seguro de refatoração

1. Identificar um problema concreto de compreensão, manutenção ou evolução, evitando refatorar apenas para satisfazer uma métrica.

2. Garantir uma rede de segurança com testes automatizados relevantes. Quando não há testes, caracterizar o comportamento existente antes da mudança.

3. Escolher uma transformação pequena e compatível com o problema observado.

4. Executar a alteração sem misturá-la à implementação de uma nova funcionalidade.

5. Compilar, executar testes e analisar o resultado após cada passo significativo.

6. Revisar nomes, responsabilidades, duplicações e dependências introduzidas pela nova estrutura.

7. Registrar alterações pequenas no controle de versão para facilitar revisão e reversão.

3.3 Principais grupos de técnicas

Composição de métodos: técnicas como Extract Method, Inline Method, Extract Variable e Substitute Algorithm tornam a intenção das operações mais evidente e reduzem a complexidade local (Refactoring.Guru, s. d.-b).

Movimentação de responsabilidades: Move Method, Move Field, Extract Class, Inline Class, Hide Delegate e Remove Middle Man ajustam os limites entre objetos e procuram equilibrar coesão e acoplamento (Refactoring.Guru, s. d.-b).

Organização de dados: Replace Data Value with Object, Encapsulate Field, Encapsulate Collection e Replace Type Code with Class tornam conceitos do domínio explícitos e concentram validações (Refactoring.Guru, s. d.-b).

Simplificação de condicionais e chamadas: Decompose Conditional, Replace Conditional with Polymorphism, Introduce Parameter Object e Preserve Whole Object reduzem decisões repetidas e interfaces difíceis de utilizar (Refactoring.Guru, s. d.-b).

Generalização: Pull Up Method, Push Down Method, Extract Interface, Collapse Hierarchy e Replace Inheritance with Delegation corrigem hierarquias que não expressam adequadamente as relações entre os conceitos (Refactoring.Guru, s. d.-b).

3.4 Exemplo de refatoração: Extract Method

O método a seguir ilustra, de forma simplificada, a aplicação de Extract Method: o subtotal e a regra de desconto aparecem no mesmo bloco e são separados em etapas nomeadas. O exemplo demonstra a técnica; isoladamente, seu tamanho não basta para afirmar que há um odor Long Method.

Código antes:

```
double total(Pedido pedido) {
    double subtotal = 0;
    for (Item item : pedido.getItens()) {
        subtotal += item.getPreco() * item.getQuantidade();
    }
    if (subtotal > 500) subtotal *= 0.9;
    return subtotal;
}
```

Código depois:

```
double total(Pedido pedido) {
    return aplicarDesconto(calcularSubtotal(pedido));
}

double calcularSubtotal(Pedido pedido) {
    double subtotal = 0;
    for (Item item : pedido.getItens()) {
        subtotal += item.getPreco() * item.getQuantidade();
    }
    return subtotal;
}

double aplicarDesconto(double subtotal) {
    if (subtotal > 500) subtotal *= 0.9;
    return subtotal;
}
```

A técnica aplicada é Extract Method. Os métodos extraídos tornam explícitas as duas responsabilidades e permitem examinar cada etapa separadamente. O exemplo não demonstra ganho de desempenho nem ausência de defeitos; testes ainda são necessários para verificar a preservação do comportamento em casos relevantes.

3.5 Testes, ferramentas e análise estática

Testes automatizados são essenciais porque permitem verificar se o comportamento observável foi preservado. Testes unitários ajudam a proteger regras locais, enquanto testes de integração e de aceitação cobrem contratos entre componentes e fluxos críticos. Uma suíte que apenas aumenta cobertura numérica, sem afirmar resultados relevantes, não oferece segurança suficiente.

Ambientes de desenvolvimento integrados automatizam operações como renomear símbolos, extrair métodos e mover classes. Ferramentas de análise estática, entre elas SonarQube, PMD e ESLint, podem apontar duplicação, complexidade, dependências e código

não utilizado. Esses alertas devem apoiar a decisão humana, pois métricas não conhecem integralmente o domínio nem as restrições arquiteturais.

3.6 Quando refatorar e quando não refatorar

A refatoração é especialmente útil antes de acrescentar uma funcionalidade, durante a correção de um defeito, em revisões de código ou quando mudanças recorrentes expõem uma estrutura inadequada. O melhor momento costuma ser próximo da necessidade real, quando o problema e os testes estão disponíveis.

Não é recomendável refatorar indiscriminadamente código estável que não precisa ser compreendido ou alterado, especialmente quando o risco operacional supera o benefício. Reescrever pode ser considerado quando a tecnologia deixou de ser suportada, o comportamento necessário mudou radicalmente ou a estrutura existente impede testes e evolução. Mesmo nesses casos, uma reescrita exige estratégia de migração e validação; começar do zero não elimina os riscos do domínio.

4 METODOLOGIA PRÁTICA: ATIVIDADE DESPLUGADA

Como forma de aproximar a teoria de odores em código-fonte e refatoração da realidade educacional, projetou-se uma dinâmica desplugada inspirada no formato tradicional de "torta na cara", mas adaptada para o contexto de ensino sem o uso de computadores. A atividade denominada "Estouro Lógico" visa fixar conceitos de refatoração de maneira interativa e colaborativa.

4.1 Objetivo da Atividade

A atividade tem por objetivo estimular o raciocínio rápido, a cooperação entre equipes e a consolidação dos conceitos de odores de código e técnicas de refatoração abordados na taxonomia de Refactoring.Guru (s. d.-a). O caráter desplugado permite o engajamento imediato dos participantes na identificação de problemas estruturais sem a barreira de um ambiente de desenvolvimento formal.

4.2 Materiais Utilizados

Para a execução da dinâmica, empregam-se os seguintes recursos:

a) Balões de gás (bexigas), representando a integridade estrutural (vidas) das equipes;

b) Suportes ou cordas para fixação dos balões junto aos grupos;

c) Cartões de desafio contendo cenários práticos de odores e refatoração;

d) Dispositivo sinalizador (sino) para controle de vez nas respostas;

e) Alfinetes para o procedimento de estouro dos balões penalizados.

4.3 Dinâmica e Regras de Execução

A condução da atividade ocorre por meio das seguintes etapas estruturadas:

1. Divisão das equipes: A turma é dividida em grupos equivalentes. Cada equipe recebe inicialmente um quantitativo fixo de balões, simbolizando sua margem de sustentabilidade estrutural.

2. Rodadas de arguição: O mediador apresenta um cenário clínico de código com um odor específico (por exemplo, Long Method ou Feature Envy). O primeiro grupo a acionar o sinalizador obtém o direito de resposta.

3. Avaliação e penalização: Caso o grupo responda corretamente indicando o odor e a técnica de refatoração adequada, adquire o direito de estourar um balão da equipe adversária. Se houver erro, a própria equipe incorre na perda de um de seus balões.

4. Critério de vitória: O processo repete-se de forma sucessiva até que reste apenas uma equipe com balões intactos, sagrando-se vencedora pelo domínio e sobrevivência estrutural de seu código simulado.

5 CONCLUSÃO

Odores em código-fonte oferecem um vocabulário prático para reconhecer estruturas que podem dificultar a compreensão, a manutenção e a evolução de sistemas. As cinco categorias analisadas mostram que esses problemas podem resultar do crescimento gradual de métodos e classes, do uso inadequado da orientação a objetos, da distribuição incorreta de responsabilidades, da permanência de elementos dispensáveis e do desequilíbrio entre acoplamento e delegação.

A presença de um odor não constitui prova automática de baixa qualidade. Ela indica a necessidade de investigar o contexto, observar o histórico de mudanças e avaliar consequências. Essa postura evita que limites numéricos ou recomendações gerais substituam o raciocínio de projeto. Também reduz o risco de aplicar abstrações desnecessárias ou transferir o problema para outra parte do sistema.

A refatoração fornece técnicas para melhorar a estrutura interna sem alterar o comportamento observável, mas sua segurança depende de passos pequenos, testes, revisão e controle de versão. Não existe técnica universal: a mesma refatoração pode resolver diferentes odores, e um odor pode admitir diversas soluções. A escolha adequada é aquela que torna a próxima mudança mais clara e menos arriscada sem criar complexidade superior ao problema original.

Conclui-se, portanto, que identificar odores e refatorar são atividades complementares, mas dependentes de diagnóstico contextual. Quando incorporadas ao desenvolvimento cotidiano, ajudam a administrar a dívida técnica e preservar a capacidade de evolução do software. Seu valor não está em produzir código considerado perfeito, mas em sustentar decisões conscientes, verificáveis e compatíveis com as necessidades reais do projeto.

REFERÊNCIAS

CAGNINI, Henry. **Introdução**. In: CAGNINI, Henry. Software Engineering. GitHub, [s. d.]. Disponível em: https://github.com/CTISM-Prof-Henry/softwareEngineering/blob/main/cap%C3%ADtulos/1.%20INTRODU%C3%87%C3%83O.md. Acesso em: 9 out. 2026.

FOWLER, Martin. **Code smell**. Martin Fowler, 9 fev. 2006. Disponível em: https://martinfowler.com/bliki/CodeSmell.html. Acesso em: 19 out. 2026.

FOWLER, Martin. **Definition of refactoring**. Martin Fowler, 1 set. 2004. Disponível em: https://martinfowler.com/bliki/DefinitionOfRefactoring.html. Acesso em: 19 out. 2026.

FOWLER, Martin. **Refatoração: aperfeiçoando o design de códigos existentes**. 2. ed. São Paulo: Novatec, 2020.

REFACTORING.GURU. **Code smells**. [S. l.], [s. d.-a]. Disponível em: https://refactoring.guru/pt-br/refactoring/smells. Acesso em: 19 out. 2026.

REFACTORING.GURU. **Refatoração**. [S. l.], [s. d.-b]. Disponível em: https://refactoring.guru/pt-br/refactoring. Acesso em: 19 out. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.